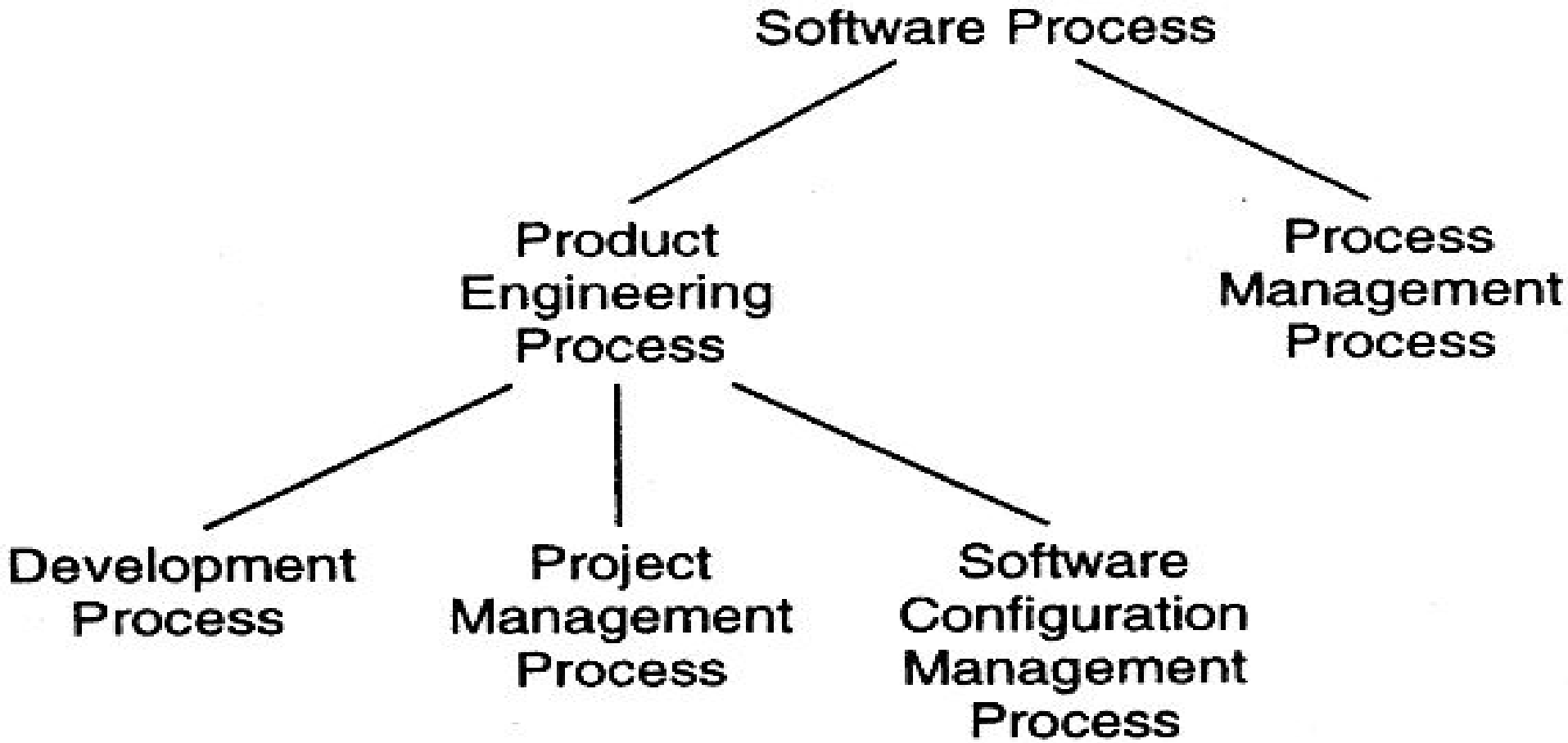
Project Management & Software Engineering

Module -1

Process and Project



Process and Project

- A Process is the sequence of steps executed to achieve a goal. Since many different goals may have to be satisfied while developing software, multiple processes are needed.
- The processes that deal with the technical and management issues of software development are collectively called the **software process**. As a software project will have to engineer a solution and properly manage the project, there are clearly two major components in a software process—a **Development process** and a **Project management process.**

Process and Project (contd..)

- The Development process specifies all the engineering activities that need to be performed,
- whereas
- the Management process specifies how to plan and control these activities so that cost, schedule, quality, and other objectives are met.

Process and Project (contd..)

- As development processes generally do not focus on evolution and changes, to handle them another process called Software configuration control process is often used.

The objective of this component process is to primarily deal with managing change, so that the integrity of the products is not violated despite changes.

Process and Project (contd..)

- These three constituent processes focus on the projects and the products and can be considered as comprising the **Product Engineering Processes**, as their main objective is to produce the desired product.

Process and Project (contd..)

- The whole process of understanding the current process, analyzing its properties, determining how to improve, and then affecting the improvement is dealt with by the **Process Management process.**

Basics of Software Life Cycle Model

- Life cycle model

A software life cycle model (also called process model) is a descriptive and diagrammatic representation of the software life cycle. A life cycle model represents all the activities required to make a software product transit through its life cycle phases.

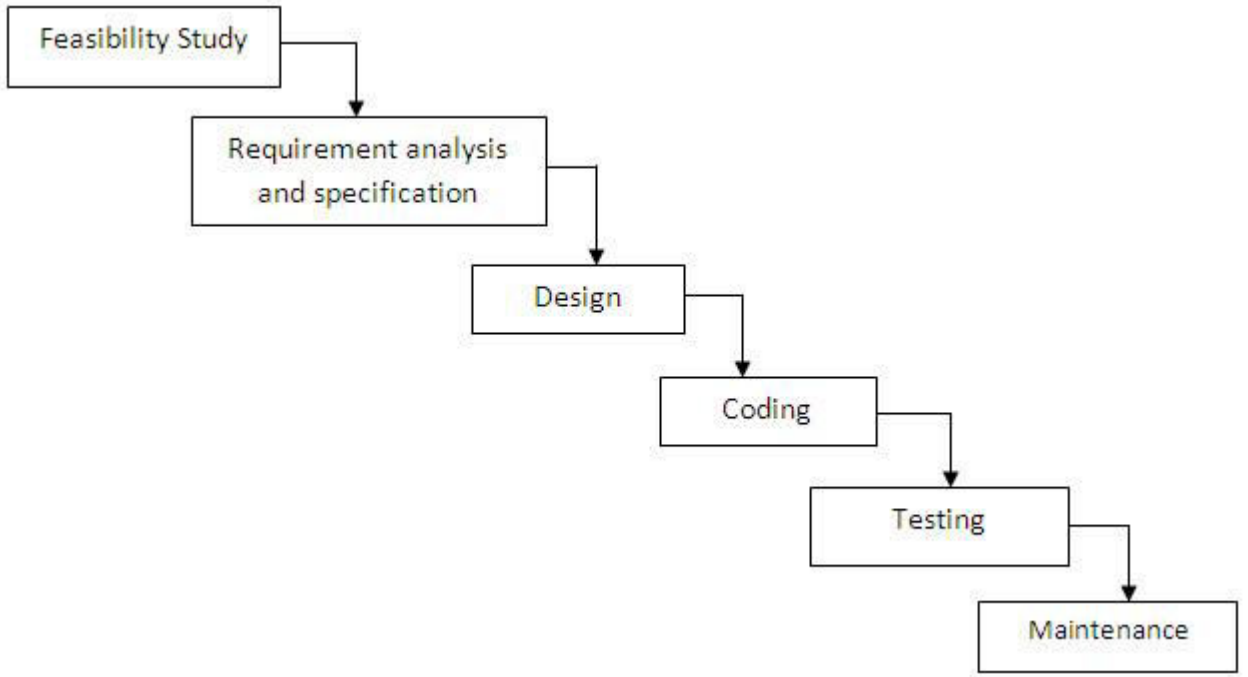
The need for a software life cycle model

- A software life cycle model defines entry and exit criteria for every phase. A phase can start only if its phase-entry criteria have been satisfied. So without software life cycle model the entry and exit criteria for a phase cannot be recognized. Without software life cycle models , it becomes difficult for software project managers to monitor the progress of the project.

Different software life cycle models

- Classical Waterfall Model
- Iterative Waterfall Model
- Prototyping Model
- Evolutionary Model
- Spiral Model

Waterfall model



Different phases of the classical waterfall model

- Feasibility Study
- Requirements Analysis& Specification
- Design
- Coding & Unit Testing
- Integration & System Testing
- Maintenance

Activities undertaken during feasibility study: -

- The main aim of feasibility study is to determine whether it would be financially and technically feasible to develop the product.
  - At first project managers or team leaders try to have a rough understanding of what is required to be done by visiting the client side. They study different input data to the system and output data to be produced by the system. They study what kind of processing is needed to be done on these data and they look at the various constraints on the behavior of the system.

- After they have an overall understanding of the problem they investigate the different solutions that are possible. Then they examine each of the solutions in terms of what kind of resources required, what would be the cost of development and what would be the development time for each solution.
- Based on this analysis they pick the best solution and determine whether the solution is feasible financially and technically. They check whether the customer budget would meet the cost of the product and whether they have sufficient technical expertise in the area of development.

Activities undertaken during requirements analysis and specification: -

- The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document them properly. This phase consists of two distinct activities, namely
- Requirements gathering and analysis, and
- Requirements specification

  The customer requirements identified during the requirements gathering and analysis activity are organized into a **SRS document**. The important components of this document are functional requirements, the nonfunctional requirements, and the goals of implementation.

Activities undertaken during design: -

- The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language. In technical terms, during the design phase the software architecture is derived from the SRS document

Activities undertaken during coding and unit testing:-

- The purpose of the coding and unit testing phase (sometimes called the implementation phase) of software development is to translate the software design into source code. Each component of the design is implemented as a program module. The end-product of this phase is a set of program modules that have been individually tested.
- During this phase, each module is unit tested to determine the correct working of all the individual modules. It involves testing each module in isolation as this is the most efficient way to debug the errors identified at this stage.

Activities undertaken during integration and system testing: -

- During the integration and system testing phase, the modules are integrated in a planned manner. During each integration step, the partially integrated system is tested and a set of previously planned modules are added to it. Finally, when all the modules have been successfully integrated and tested, system testing is carried out. The goal of system testing is to ensure that the developed system conforms to its requirements laid out in the SRS document.

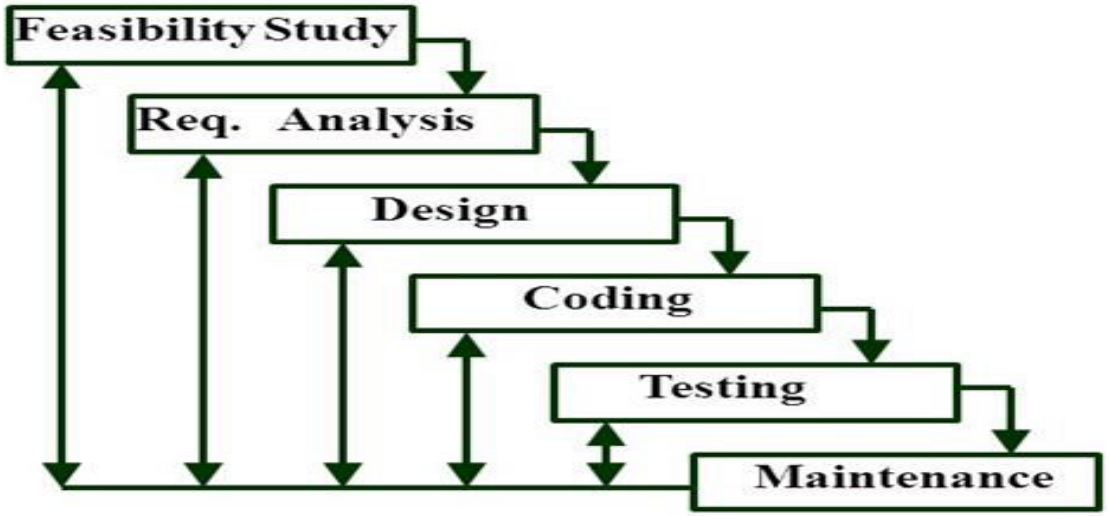
Activities undertaken during maintenance: -

- Maintenance involves performing any one or more of the following three kinds of activities:
  - Correcting errors that were not discovered during the product development phase. This is called corrective maintenance.
  - Improving the implementation of the system, and enhancing the functionalities of the system according to the customer's requirements. This is called perfective maintenance.
  - Porting the software to work in a new environment. For example, porting may be required to get the software to work on a new computer platform or with a new operating system. This is called adaptive maintenance.
  -

Shortcomings of the classical waterfall model

- The classical waterfall model is an idealistic one since it assumes that no development error is ever committed by the engineers during any of the life cycle phases. However, in practical development environments, the engineers do commit a large number of errors in almost every phase of the life cycle. These defects usually get detected much later in the life cycle.
- Once a defect is detected, the engineers need to go back to the phase where the defect had occurred and redo some of the work done during that phase and the subsequent phases to correct the defect and its effect on the later phases. Therefore, in any practical software development work, it is not possible to strictly follow the classical waterfall model.

Iterative Waterfall Model



Iterative Waterfall Model

- In the iterative development model, software is developed in iterations, each iteration resulting in a working software system. This model does not require all requirements to be known at the start, allows feedback from earlier iterations for next ones, and reduces risk as it delivers value as the project proceeds.
- The basic idea is that the software should be developed in increments, each increment adding some functional capability to the system until the full system is implemented.

Prototyping Model

- In the prototyping model, a prototype is built before building the final system, which is used to further develop the requirements leading to more stable requirements. This is useful for projects where requirements are not clear.
- A prototype is a toy implementation of the system. A prototype usually exhibits limited functional capabilities, low reliability, and inefficient performance compared to the actual software.

Need for a prototype in software development

- There are several uses of a prototype. An important purpose is to illustrate the input data formats, messages, reports, and the interactive dialogues to the customer. This is a valuable mechanism for gaining better understanding of the customer's needs:
  - • how the screens might look like
  - • how the user interface would behave
  - • how the system would produce outputs

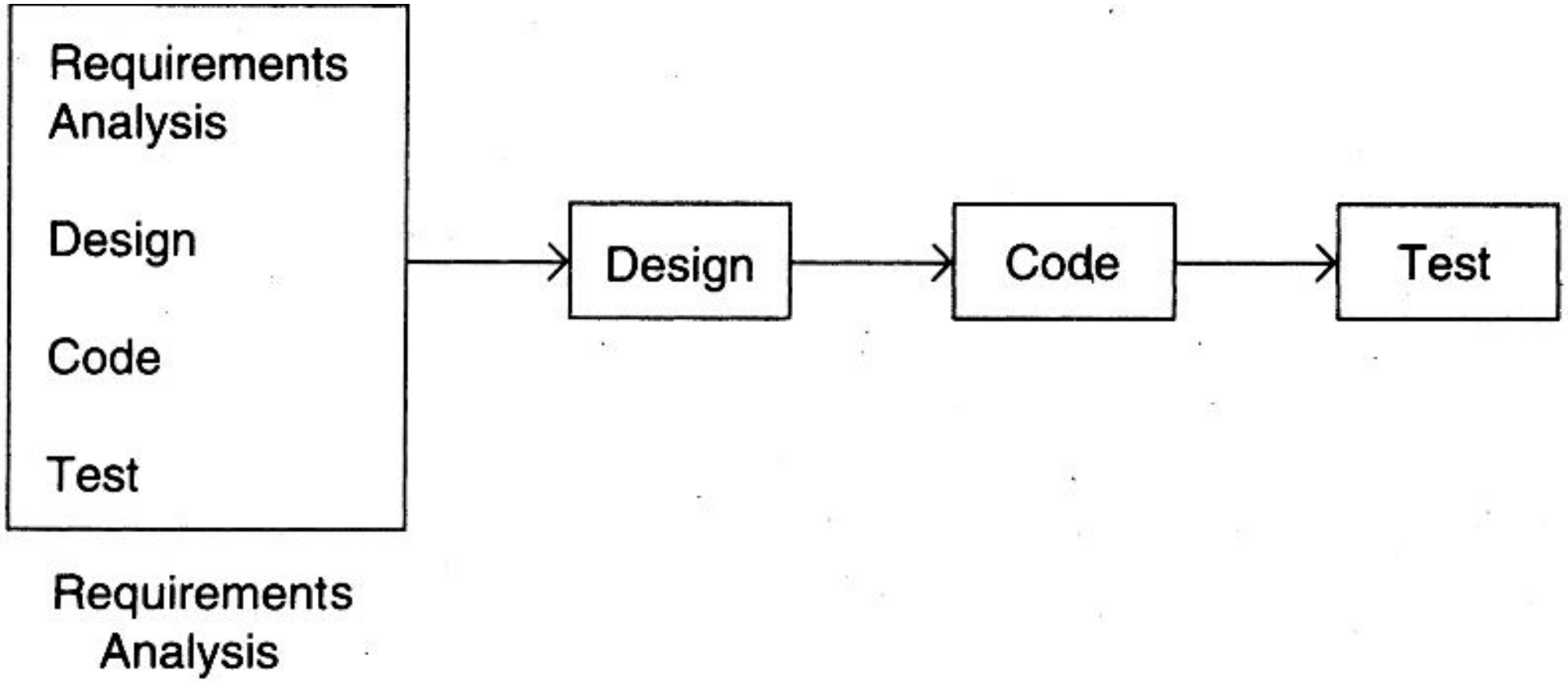
Need for a prototype in software development (contd..)

- Another reason for developing a prototype is that it is impossible to get the perfect product in the first attempt. Many researchers and engineers advocate that if you want to develop a good product you must plan to throw away the first version. The experience gained in developing the prototype can be used to develop the final product.

Need for a prototype in software development (contd..)

- A prototyping model can be used when technical solutions are unclear to the development team. A developed prototype can help engineers to critically examine the technical issues associated with the product development. Often, major design decisions depend on issues like the response time of a hardware controller, or the efficiency of a sorting algorithm, etc. In such circumstances, a prototype may be the best or the only way to resolve the technical issues.

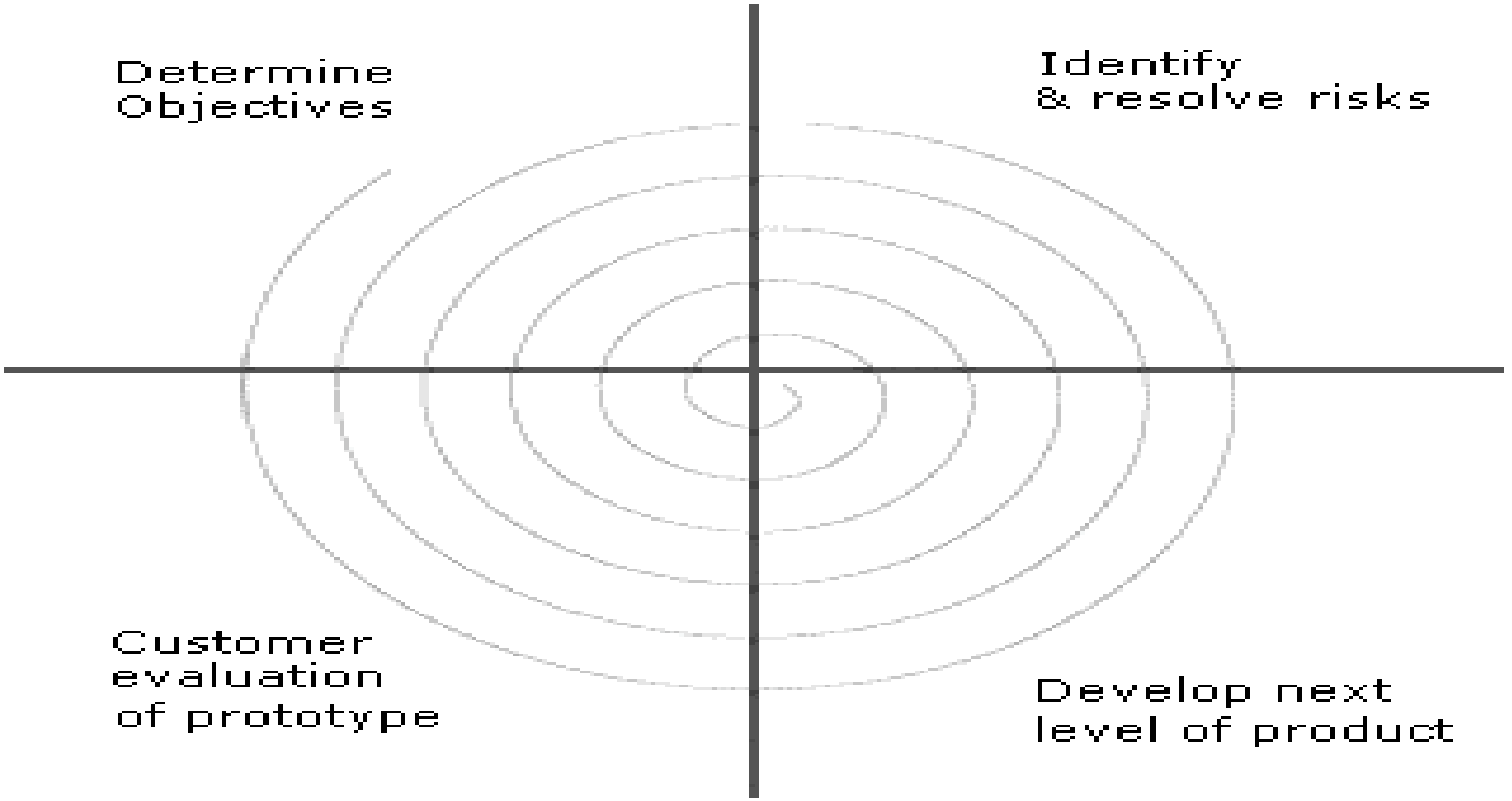
Prototyping Model



Spiral model

- The Spiral model of software development is shown in fig. The diagrammatic representation of this model appears like a spiral with many loops. The exact number of loops in the spiral is not fixed. Each loop of the spiral represents a phase of the software process. For example, the innermost loop might be concerned with feasibility study. The next loop with requirements specification, the next one with design, and so on. Each phase in this model is split into four sectors (or quadrants) as shown in fig. The following activities are carried out during each phase of a spiral model..

Spiral model



Spiral model

- **First quadrant (Objective Setting)**
  - • During the first quadrant, it is needed to identify the objectives of the phase. Examine the risks associated with these objectives.
- - **Second Quadrant (Risk Assessment and Reduction)**
  - • A detailed analysis is carried out for each identified project risk.
  - • Steps are taken to reduce the risks. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.

Spiral model

- **Third Quadrant (Development and Validation)**
  - • Develop and validate the next level of the product after resolving the identified risks.
- - **Fourth Quadrant (Review and Planning)**
  - • Review the results achieved so far with the customer and plan the next iteration around the spiral.
  - • Progressively more complete version of the software gets built with each iteration around the spiral.

Circumstances to use spiral model

- The spiral model is called a meta model since it encompasses all other life cycle models. Risk handling is inherently built into this model. The spiral model is suitable for development of technically challenging software products that are prone to several kinds of risks. However, this model is much more complex than the other models – this is probably a factor deterring its use in ordinary projects.

Extreme Programming and Agile Processes

Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach. Agile approaches are based on some common principles, some of which are

– Working software is the key measure of progress in a project.

– For progress in a project, therefore, software should be developed and delivered rapidly in small increments.

- Even late changes in the requirements should be entertained (small-increment model of development helps in accommodating them).

Extreme Programming and Agile Processes(contd…)

– Face-to-face communication is preferred over documentation.

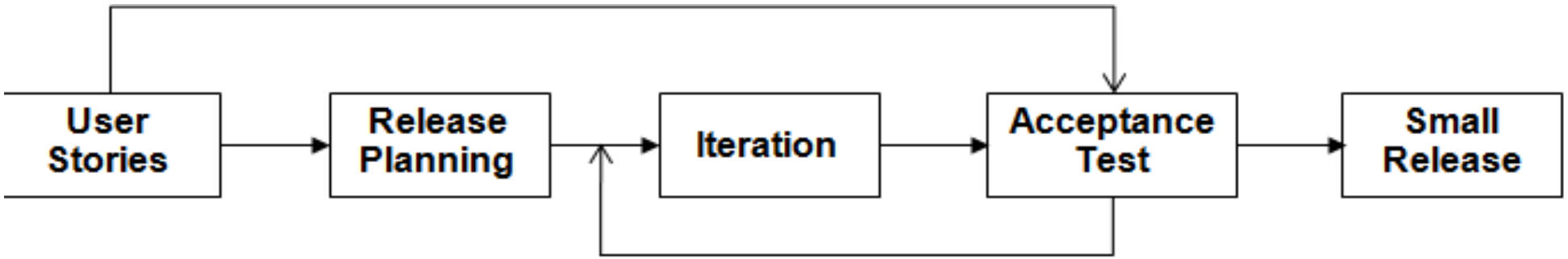
- Continuous feedback and involvement of customer is necessary for developing good-quality software.

– Simple design which evolves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.

– The delivery dates are decided by empowered teams of talented individuals (and are not dictated).

Extreme programming (XP) is one of the most popular and well-known approaches in the family of agile methods.

Extreme Programming



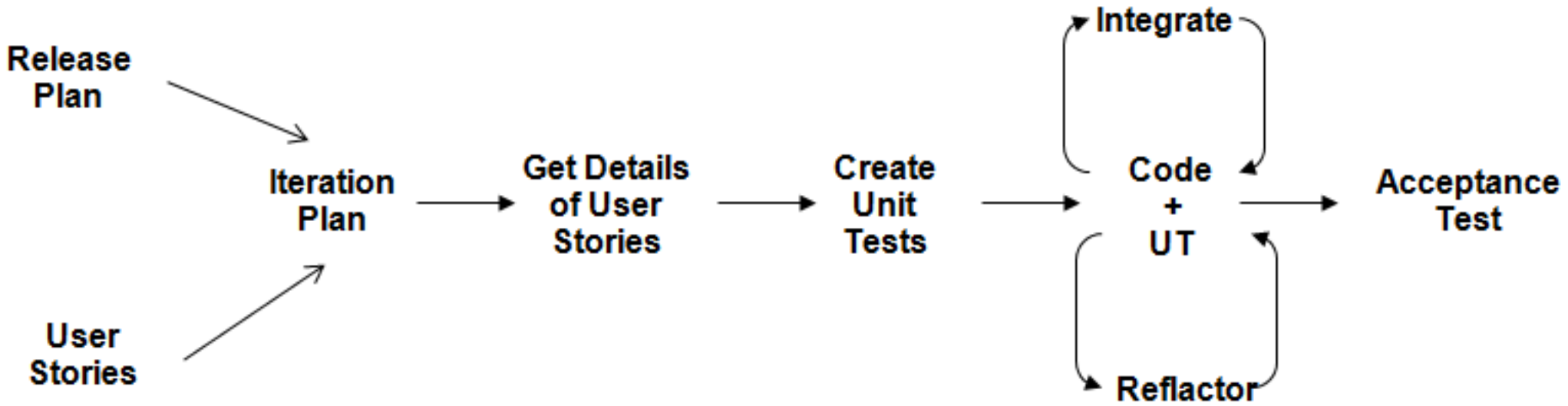
Extreme Programming (contd..)

- An extreme programming project starts with user stories which are short (a few sentences) descriptions of what scenarios the customers and users would like the system to support. Each story is written on a separate card, so they can be flexibly grouped.
- The empowered development team estimates how long it will take to implement a user story. The estimates are rough, generally stated in weeks. Using these estimates and the stories, release planning is done which defines which stories are to be built in which system release, and the dates of these releases.

Extreme Programming (contd..)

- Acceptance tests are also built from the stories, which are used to test the software before the release. Bugs found during the acceptance testing for an iteration can form work items for the next iteration.
- Writing unit tests before programming and keeping all of the tests running at all times. The unit tests are automated and eliminates defects early, thus reducing the costs.
- Starting with a simple design just enough to code the features at hand and redesigning when required.
- Programming in pairs (called pair programming), with two programmers at one screen, taking turns to use the keyboard. While one of them is at the keyboard, the other constantly reviews and provides inputs.
- Integrating and testing the whole system several times a day.
- Putting a minimal working system into the production quickly and upgrading it whenever required.
- Keeping the customer involved all the time and obtaining constant feedback.

An iteration in XP



Assignment – Module 1

1.Compare the different lifecycle models you have studied

2. Identify the Strength and Weakness of each model

Last Date of Submission : 18/07/2019